On problems in the real estate market
Technical assessment: Junior Data Scientist

Muriz Serifovic*
Monday 22nd July, 2019

***Abstract*— This paper proposes a set of solutions to the problems posed by ImmoSparrow[1]. Machine learning can play an important role in the real estate market to predict the price of a property. Herein, I provide an overview of a variety of different regression models for house price prediction. For illustration purposes, I empirically compare the effectiveness of the models on the popular Boston housing dataset [Harrison Jr and Rubinfeld, 1978]. Furthermore, to address the second problem, I give an outline of image classification and object recognition methods for real estate advertisements and draw a comparison between third-party APIs. My source code is publicly available at `https://github.com/Murgio/real-estate-problems`.**

I. House Price Prediction

ACCURATE housing predictions are essential for all residential property industry respondents. An analysis of the literature on the housing market indicates a key research trend: the use of hedonic-based regression approaches [Adair et al., 1996, Selim, 2009, Meese and Wallace, 2003] for house price prediction. The problem here is of similar nature:

Problem statement. *Suppose we want to estimate the price of real estate. We know the target variable price of around 100,000 properties and around 10 features such as living space, land area, village, noise, etc. Which ML methods could be suitable for this task?*

In order to make an informed decision about the choice of ML methods, one should examine the dataset first. Exploratory Data Analysis (EDA) is an important and recommended first step prior to the training of a machine learning model. As it turns out, the problem lends itself naturally to a linear regression model. To justify my choice of a linear model, I refer the reader to appendix II.

The principle is to assume a priori that there exists some sort of an underlying linear relationship between house prices and housing characteristics. One can presume for example that more rooms are more valued (Fig. 1). Houses with more rooms are usually larger and can fit more people, so that they cost more money is a reasonable assumption.

Throughout this section, I will extensively use the NumPy, matplotlib and the scikit-learn library [Pedregosa et al., 2011], which offers a user-friendly interface for using discussed algorithms efficiently and productively.

*Correspondence email address: muriz.se@gmail.com
[1] `https://www.immosparrow.ch`

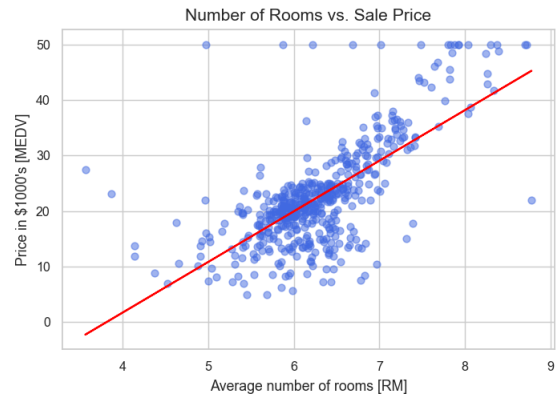

Fig. 1: Drawing a simple regression line between average number of rooms and sale price already explains 69% of variance in sale price.

Data source and pre-processing: It seemed interesting to me to empirically investigate the predictive power of the models, so I replaced the proposed fictive dataset with the similar Boston housing dataset, which contains information about houses in the suburbs of Boston. Of note, the dataset is similar only in terms of feature size, whereas the number of properties differs enormously, i.e., there exist only 506 properties in the Boston housing dataset in comparison to 10^5 properties in the fictive set (see appendix I for more details). I shuffle the data into a random order to remove any bias in the ordering of the dataset. In order to assess the models performance on unseen data, I split the dataset into training and testing sets with 80% of the samples and the remaining 20% respectively.

A. Linear regression

Starting out as my baseline model, the linear regression model is, in its simplest form with one explanatory variable, defined as follows:

$$y = \beta_0 + \beta_1 x_1 \tag{1}$$

where y is the target price, x_1 is the data for a single feature, β_0 represents the y axis intercept and β_1 is the coefficient of the explanatory variable.

The linear model either assumes that the regression function is linear, or that the linear model is a reasonable approximation.

B. Ordinary Least Squares regression (OLS)

To start improving the linear predictions by incorporating the influence of additional variables, I extend the definition to fit the problem more neatly with multiple explanatory variables, namely

$$f(X) = \beta_0 + \sum_{j=1}^{p} X_j \beta_j \tag{2}$$

Typically, as in this case, we have a set of training data $(x_1, y_1) \ldots (x_N, y_N)$ from which to estimate the parameters β. Each $x_i = (x_{i1}, x_{i2}, \ldots, x_{ip})^T$ is a vector of feature measurements for the ith case. The most popular estimation method is least squares, in which we pick the coefficients $\beta = (\beta_0, \beta_1, \ldots, \beta_p)^T$ to minimize the residual sum of squares, namely

$$\begin{aligned} \mathrm{RSS}(\beta) &= \sum_{i=1}^{N} (y_i - f(x_i))^2 \\ &= \sum_{i=1}^{N} \left(y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j \right)^2 \end{aligned} \tag{3}$$

This simple model has several shortcomings. The first shortcoming is that the least squares estimates often have low bias but large variance. Sometimes the accuracy can be improved by reducing or setting certain coefficients to zero. In doing this I can sacrifice a small amount of bias to decrease the variance of the forecasted values, and therefore improve the overall model accuracy. There could also be strong relationships between some of the explanatory variables, for that reason regularization techniques will be critical for controlling this multicollinearity.

The second shortcoming is that least squares estimates are highly sensitive to outliers because of the quadratic weight of the residuals, which means that outlying points can greatly affect the fit. Therefore, during the data pre-processing stage, I remove outliers after manually inspecting the scatter splot.

C. Introducing Regularization

Regularization is one approach to tackle the problem of overfitting by adding additional information, and thereby shrinking the parameter values of the model to induce a penalty against complexity. The most popular approaches to regularized linear regression are Ridge regression, LASSO and the Elastic Net method.

1) Ridge regression: Ridge regression, also known as $L2$ regularization[2], seeks to minimize the following equation:

$$\sum_{i=1}^{N} \left(y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j \right)^2 + \lambda \sum_{j=1}^{p} \beta_j^2 \tag{4}$$

where increasing the value of the hyperparameter λ increases the regularization strength and the amount of shrinkage.

[2]Mathematically, Ridge penalizes the L2 norm of the coefficients

Model	R^2	RMSE	10-fold cross validation
One explanatory variable	0.543	5.790	0.516
All explanatory variables	0.668	4.931	0.699
Ridge regression	0.668	4.392	0.698
Lasso regression	0.669	4.927	0.664
Elastic net	0.673	4.898	0.658
Quadratic polynomial	0.807	3.765	0.762
Cubic polynomial	0.872	3.403	**0.851**
Random forest regression	**0.877**	**3.032**	0.464

TABLE I: Fitting the model with one explanatory variable yields the worst predictor, whereas fitting the random forest regressor gives the best house price estimator.

2) LASSO: An alternative to Ridge for regularizing linear regression is Lasso. As with ridge regression, using the lasso also restricts coefficients to be close to zero, but in a slightly different way, called L1 regularization[3].

$$\sum_{i=1}^{N} \left(y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j \right)^2 + \lambda \sum_{j=1}^{p} |\beta_j| \tag{5}$$

3) Elastic net: Elastic net combines the penalties of ridge regression and lasso to get the best of both worlds. Elastic net aims at minimizing the following equation:

$$\frac{\sum_{i=1}^{n} (y_i - x_i'\beta)^2}{2n} + \lambda \left(\frac{1-\alpha}{2} \sum_{j=1}^{p} \beta_j^2 + \alpha \sum_{j=1}^{p} |\beta_j| \right) \tag{6}$$

where α is the mixing parameter between ridge ($\alpha = 0$) and lasso ($\alpha = 1$).

D. Polynomial regression

In the previous sections, I assumed a linear relationship between explanatory and response variables. One way to account for the violation of linearity assumption is to use a polynomial regression model by adding polynomial terms:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_1^2 + \ldots + \beta_d x_i^d \tag{7}$$

where d is the degree of the polynomial. For the empirical results, I consider two models bounded by polynomials of the second and third degree respectively (see Fig. 2 for a comparison of $\{1, 2, 3\}^{th}$ degree polynomial fit).

E. Random forest regression

The last model I am taking into consideration[4] is an ensemble technique that combines multiple decision trees [Breiman et al., 1984, Hunt et al., 1966, Song and Ying, 2015, Loh, 2014]. A random forest generalizes better than an individual decision tree owing to the randomness that helps to reduce the variation in the model. Another advantage of random forests is that the outliers in the dataset are less susceptible and involve little parameter tuning.

[3]The lasso penalizes the L1 norm of the coefficient vector

[4]Further possible models: ANN, SVM regressor, AdaBoost, Gradient Boosting, XGBoost, Model ensembles

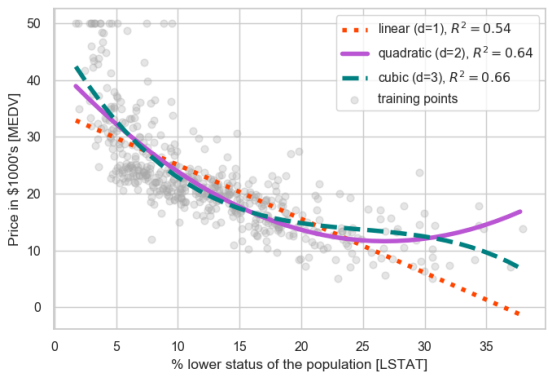

Fig. 2: Second degree (quadratic) and third degree (cubic) polynomials compared to a linear fit.

F. Model performance

Without quantifying the performance of a particular model during training and testing, it is difficult to measure the quality of the model. Typically, this is done with certain performance metrices, be they by calculating a certain type of failure, the fitness or other useful measures. This section addresses the problem $1b$, which is given as follows:

Problem statement. *We have decided to test a model that has several tuning parameters for the price estimation of real estate. How do you proceed in order to find out as quickly as possible whether the model has a good performance and how the tuning parameters have to be set?*

1) Metrics: A typical performance measure for regression problems is the Root Mean Square Error (RMSE). It gives an idea of how much error the system typically makes in its predictions, with a higher weight for large errors.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{8}$$

Further[5], I will be calculating the coefficient of determination, R^2, which can be understood as a standardized version of the mean squared error.

2) Tuning hyperparameters: Conducting the search of the best hyperparameters could be achieved by trial and error, however in the current setting for regression analysis the scikit-learn library provides us with two popular ways as a remedy.

Grid Search: Grid search is an exhaustive search over a subset of the parameter space.

Random Search: If we're interested to search over the parameter space in random manner, random search is the preferable way.

[5]Further possible metrics include F value, mean absolute error, mean squared error and adjusted R^2

II. Results

For reproducibility, I choose in all models a NumPy [Oliphant, 2006] seed of 42. In Table I, I present a summary of the results.

III. Object recognition

One potential limit a regression model might suffer from is that even after tying all data sources together, such as the traditional numerical, categorical, and text data associated with a real estate listing, the model can give an incomplete picture of an accurate property valuation. Although traditional ML models are quick and cheap, they can't truly take into account the quality and condition of a property without really looking at the property, thereby limiting the accuracy of the tool in the financial services industry. Taking pictures of a real estate into account for price prediction might yield more accurate estimates. The problem here is as follows:

Problem statement. *We have an image database with photos from real estate advertisements. Now we would like to carry out an object recognition for each picture, e.g. to detect a house or a bathroom. What ML method would you use and why?*

The definition of the question seems ambiguous to me. It is not very clear whether for example boundary boxes or maybe even pixel-wise segmentation maps are being desired or not. I will focus on image classification and object recognition, the two most popular branches in computer vision, namely narrowing my view further down to deep learning[6] and Convolutional Neural Networks [LeCun et al., 1989, 1990, LeCun et al., 1998, Fukushima, 1979, 1981, 2011] which yield state-of-the-art results in competitions such as ILSVRC [Russakovsky et al., 2015] , PASCAL VOC [Everingham et al., 2015], and Microsoft COCO [Lin et al., 2014].

A. Image classification:

Understanding what an image contains has received extensive attention in research over the last decades. The image classification problem requires determining the category (class) that the image belongs to. A list of the most popular architectures in image classification is listed below:

1) ***VGG Net*** *(2014) [Simonyan and Zisserman, 2014]:* VGG achieved the state-of-the-art accuracy on ILSVRC classification and localization tasks (in 2014). The VGG-16 has 13 convolutional layers with 3 fully connected layers.

2) ***GoogLe Net*** *(2015) [Szegedy et al., 2015]:* Google's network consists of a carefully crafted design called inception module that allows for increasing the depth and width of the network while keeping the computational budget constant.

[6]For a survey on deep learning in neural networks, see [Schmidhuber, 2015]

*3) **Residual Network** (2016) [He et al., 2016]:* Resnet, consisting of several basic residual blocks, was developed with the intent of designing ultra-deep networks that did not suffer from the vanishing gradient problem that predecessors had.

B. Object recognition:

Object detection is the procedure of determining the instance of the class to which the object belongs and estimating the location of the object by outputting the bounding box around the object. Detecting single instance of class from image is called as single-class object detection, whereas detecting the classes of all objects present in the image is known as multi-class object detection. Deep CNNs have been extensively used for object detection and following is a list of popular architectures:

*1) **R-CNN** (2014) [Girshick et al., 2014]:* R-CNN creates bounding boxes, or region proposals, using a process called selective search and proposes 2000 regions of interest.

*2) **You Only Look Once** (2016) [Redmon et al., 2016]:* YOLO divides up the image into a grid of 13×13 cells: each of these cells is responsible for predicting 5 bounding boxes.

Steps in the process include but are not limited to:

1) Manual tagging the entire dataset with the regions of interest in each image
   a) Splitting into training and testing data
2) Training the R-CNN or YOLO model
3) Computing and evaluating the testing results

IV. Libraries

To this date, the list of deep learning frameworks available to implement image classification or object recognition architectures is extensive. Some of them are: **TensorFlow** [Abadi et al., 2016], **PyTorch** [Paszke et al., 2017], **Keras** [Chollet et al., 2015], **Theano** [Theano Development Team, 2016] and Microsoft Cognitive Toolkit **CNTK** [Yu et al., 2014]. Every listed framework supports Python.

V. Third-party API

Training large scale networks demands high-efficient computing resources such as GPU or TPU-clusters, large annotated datasets and expert domain knowledge. If any of those constraints can not be satisfied and an in-house training of models is not feasible, one can resort to third-party APIs. What follows is a list of candidates: Clarifai, Google Cloud Vision API, Microsoft Cognitive Service, IBM Watson Vision Recognition Service, Amazon Rekognition and CloudSight.

Going one step further, there exist APIs for the real estate domain, such as FoxyAI and restb.ai.

VI. Conclusions

For house price prediction, I gave an overview of regression methods such as linear regression and random forests. The proposed methods were evaluated on the Boston housing dataset to empirically compare their predictive effectiveness. In Section III I offered an outline on ConvNets and their implications in image classification and object recognition. Further, frameworks and third-party APIs were discussed in Section IV and V respectively.

VII. Source Code

Source code for model implementations and experiments is available at `https://github.com/Murgio/real-estate-problems`.

References

Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. Tensorflow: A system for large-scale machine learning. In *OSDI*, volume 16, pages 265–283, 2016.

A. S. Adair, J. N. Berry, and W. S. McGreal. Hedonic modelling, housing submarkets and residential valuation. *Journal of Property Research*, 13(1):67–83, 1996. doi: 10.1080/095999196368899.

Leo Breiman, Jerome H Friedman, Richard A Olshen, and Charles J Stone. Classification and regression trees. belmont, ca: Wadsworth. *International Group*, 432: 151–166, 1984.

François Chollet et al. Keras. `https://github.com/fchollet/keras`, 2015.

Mark Everingham, SM Ali Eslami, Luc Van Gool, Christopher KI Williams, John Winn, and Andrew Zisserman. The pascal visual object classes challenge: A retrospective. *International journal of computer vision*, 111(1):98–136, 2015.

Kunihiko Fukushima. Neural network model for a mechanism of pattern recognition unaffected by shift in position-neocognitron. *IEICE Technical Report, A*, 62(10): 658–665, 1979.

Kunihiko Fukushima. Neocognitron–a self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *NHK Ł*, (15):p106–115, 1981.

Kunihiko Fukushima. Increasing robustness against background noise: Visual pattern recognition by a neocognitron. *Neural networks*, 24(7):767–778, 2011.

Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 580–587, 2014.

David Harrison Jr and Daniel L Rubinfeld. Hedonic housing prices and the demand for clean air. *Journal of environmental economics and management*, 5(1):81–102, 1978.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

EB Hunt, J Marin, and P Stone. Experiments in induction academic press. *New York*, 1966.

Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. Backpropagation applied to handwritten zip code recognition. *Neural Comput.*, 1(4):541–551, December 1989. ISSN 0899-7667. doi: 10.1162/neco.1989.1.4.541. URL http://dx.doi.org/10.1162/neco.1989.1.4.541.

Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, Nov 1998. ISSN 0018-9219. doi: 10.1109/5.726791.

Yann LeCun, Bernhard E Boser, John S Denker, Donnie Henderson, Richard E Howard, Wayne E Hubbard, and Lawrence D Jackel. Handwritten digit recognition with a back-propagation network. In *Advances in neural information processing systems*, pages 396–404, 1990.

Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *European conference on computer vision*, pages 740–755. Springer, 2014.

Wei-Yin Loh. Fifty years of classification and regression trees. *International Statistical Review*, 82(3):329–348, 2014.

Richard Meese and Nancy Wallace. House price dynamics and market fundamentals: The parisian housing market. *Urban Studies*, 40(5-6):1027–1045, 2003. doi: 10.1080/0042098032000074308. URL https://doi.org/10.1080/0042098032000074308.

Travis Oliphant. NumPy: A guide to NumPy. USA: Trelgol Publishing, 2006. URL http://www.numpy.org/. [Online; accessed July 22, 2019].

Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in PyTorch. In *NIPS Autodiff Workshop*, 2017.

F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788, 2016.

Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. Imagenet large scale visual recognition challenge. *International journal of computer vision*, 115(3):211–252, 2015.

Jürgen Schmidhuber. Deep learning in neural networks: An overview. *Neural networks*, 61:85–117, 2015.

Hasan Selim. Determinants of house prices in turkey: Hedonic regression versus artificial neural network. *Expert Systems with Applications*, 36 (2, Part 2):2843 – 2852, 2009. ISSN 0957-4174. doi: https://doi.org/10.1016/j.eswa.2008.01.044. URL http://www.sciencedirect.com/science/article/pii/S0957417408000596.

Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

Yan-Yan Song and LU Ying. Decision tree methods: applications for classification and prediction. *Shanghai archives of psychiatry*, 27(2):130, 2015.

Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1–9, 2015.

Theano Development Team. Theano: A Python framework for fast computation of mathematical expressions. *arXiv e-prints*, abs/1605.02688, May 2016. URL http://arxiv.org/abs/1605.02688.

Dong Yu, Adam Eversole, Mike Seltzer, Kaisheng Yao, Brian Guenter, Oleksii Kuchaiev, Frank Seide, Huaming Wang, Jasha Droppo, Zhiheng Huang, Geoffrey Zweig, Chris Rossbach, and Jon Currey. An introduction to computational networks and the computational network toolkit. *Microsoft Technical Report*, 2014.

Appendix I
Description of the Dataset

The database used in this paper was collected by D. Harrison and D.L. Rubinfeld in 1978. It involves 506 properties of which the features are summarized as shown in Table II.

Appendix II
Exploratory Data Analysis (EDA)

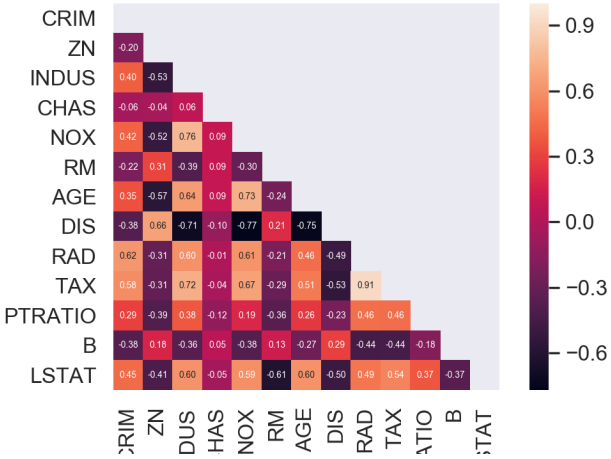

Fig. 3: Positive correlating features are shown in light red tones. (Best seen in color).

I will generate a correlation matrix (Fig. 3) to quantify the linear relationship between the features. Intuitively, the correlation matrix can be interpreted as a covariance matrix. The correlation matrix is actually the same as the covariance matrix calculated from standardized data. The matrix contains the Pearson product-moment correlation coefficients (Pairson's r), which measures linear dependency between couples of features. The correlation coefficients are bounded to the range -1 and 1. Two features have a perfect positive correlation if $r = 1$, no correlation if $r = 0$, and a perfect negative correlation if $r = -1$, respectively. To fit a linear regression model, one is interested in those features that have a high correlation with the target variable.

Name of attribute	Description
CRIM	per capita crime rate by town
ZN	proportion of residential land zoned for lots over 25,000 sq.ft.
INDUS	proportion of non-retail business acres per town
CHAS	Charles River dummy variable (= 1 if tract bounds river; 0 otherwise)
NOX	nitric oxides concentration (parts per 10 million)
RM	average number of rooms per dwelling
AGE	proportion of owner-occupied units built prior to 1940
DIS	weighted distances to five Boston employment centres
RAD	index of accessibility to radial highways
TAX	full-value property-tax rate per $10,000
PTRATIO	pupil-teacher ratio by town
B	1000(Bk - 0.63)^2 where Bk is the proportion of blacks by town
LSTAT	% lower status of the population
MEDV	Median value of owner-occupied homes in $1000's

TABLE II: Description of the Boston Housing Dataset.